

THE BEGINNER'S GUIDE — START HERE

Learn **Niko**, one small step at a time

Niko is a programming language that reads like plain English. This book will take you from zero to writing your own little games — no experience needed, just curiosity.

What's inside

1. [What is Niko? + getting set up](#)
2. [Your first program: saying hello](#)
3. [Boxes with names: variables](#)
4. [Talking to the human: **ask**](#)
5. [Making decisions: **if**](#)
6. [Round and round: loops](#)
7. [Your own commands: functions](#)
8. [Shopping lists: lists](#)
9. [Little bundles: records](#)
10. [Playing with text](#)
11. [Numbers and math](#)
12. [When things go wrong: errors](#)
13. [Borrowing code: **use**](#)
14. [The toy box: the standard library](#)
15. [Three machines, one language](#)
16. [Mini projects: build three things](#)
17. [Where to go next](#)
18. [The niko2 toolbox](#)

CHAPTER 1

What is Niko? (And how do I get it?)

A **programming language** is a way of giving instructions to a computer. Most of them look like secret code. **Niko is different**: it reads like plain English. If you can read this sentence, you can already read Niko.



peek.niko

```
if the score is bigger than 10:
    say "You win!"
otherwise:
    say "Try again!"
```

See? No curly braces, no semicolons, no mystery. Just words.

Getting set up — pick your way

There are three ways to play with Niko. Any one of them is enough for this whole book.

Way 1 — VS Code (recommended). If you use Visual Studio Code, install the extension `niko-lang.niko` from the Marketplace (search for "Niko"). It colours your code and helps you write it.

Way 2 — the `niko2` tool. Niko comes as a little program called `niko2`. You can find it inside the Niko release zip file. Unzip it somewhere, open a terminal in that folder, and type:

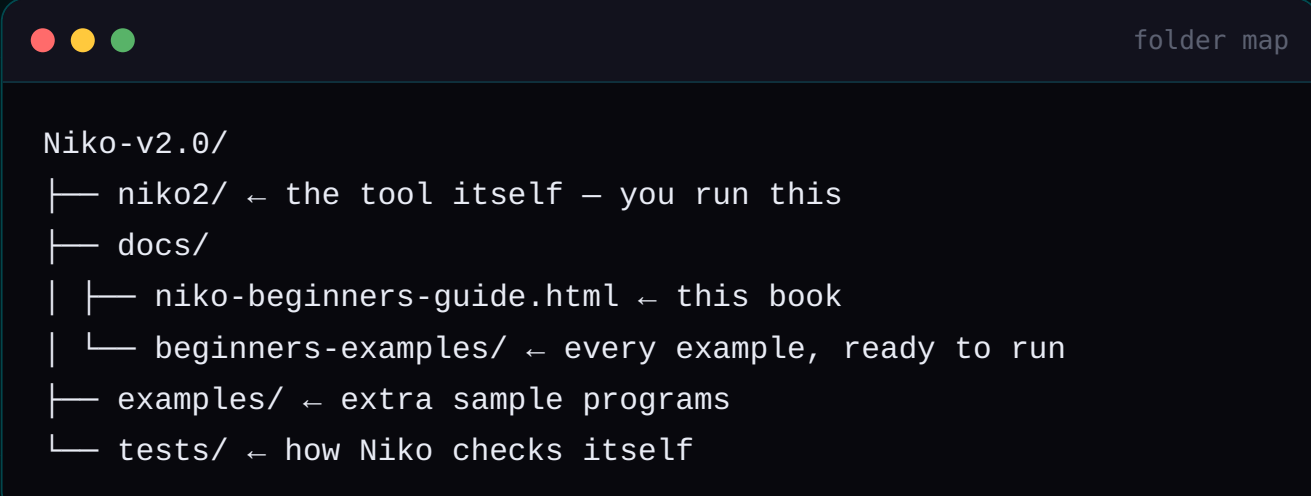
 terminal

```
$ python3 -m niko2 --help
usage: niko2 [-h] ...
Niko 2 compiler/interpreter
(a list of all the things niko2 can do appears here)
```

If you see that list, you're ready. Every example in this book runs with `python3 -m niko2 run myfile.niko`.

What's inside the folder?

Unzipping gives you a folder full of files. Here is the map — the four places that matter to you:



```
Niko-v2.0/  
├─ niko2/ ← the tool itself – you run this  
├─ docs/  
│ └─ niko-beginners-guide.html ← this book  
│ └─ beginners-examples/ ← every example, ready to run  
├─ examples/ ← extra sample programs  
└─ tests/ ← how Niko checks itself
```

`niko2/` is the language. `docs/beginners-examples/` is your playground — open any file there, read it, run it, change it, break it.

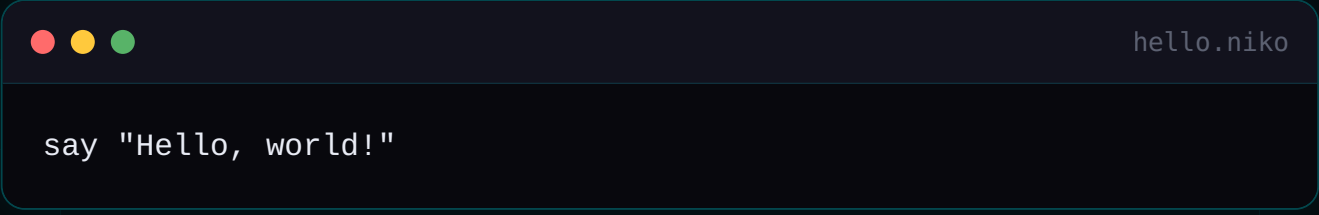
Way 3 — the playground. There is a Niko playground that runs right in your web browser — nothing to install at all. Your code is turned into WebAssembly and runs on the page.

Niko says: Hi! I'm Niko, your guide. I'll pop up with hints as we go. Don't worry about memorizing everything — programmers look things up all the time. The goal is to *play*.

CHAPTER 2

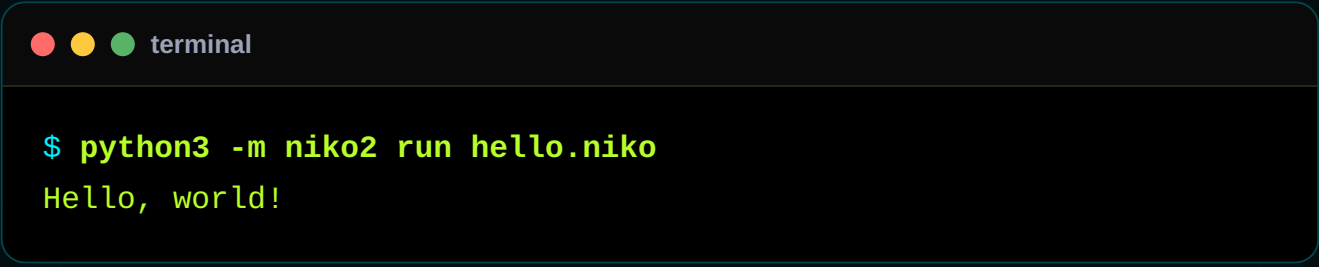
Your first program: saying hello

Every programmer's first program makes the computer say hello. Let's do it. Open a file called `hello.niko` and type exactly this:



```
say "Hello, world!"
```

One line! `say` is Niko's way of printing words on the screen. Now run it:



```
$ python3 -m niko2 run hello.niko
Hello, world!
```

Niko says: The computer did *exactly* what you told it. That's the whole secret of programming: giving very clear, very small instructions. You're already doing it!

Try it yourself

Change the words inside the quotes to your own name and run it again. What happens if you add a second line, like `say "How are you?"`? (Computers read top to bottom — predict the order, then check!)

CHAPTER 3

Boxes with names: variables

A **variable** is like a little labelled box. You put something in it, give the box a name, and later you can look inside. In Niko you make one with `set`:



variables.niko

```
set name: text to "Niko"  
set cookies: number to 3  
say name  
say cookies
```



terminal

```
$ python3 -m niko2 run variables.niko  
Niko  
3  
5
```

name

"Sam"

score

7

Two variables: a box called **name** holding "Sam", and a box called **score** holding 7.

You can put a **type** on the box — a promise about what goes inside. `text` means words, `number` means numbers. And you can change what's inside later with another `set`:



variables.niko

```
set cookies to cookies + 2  
say cookies
```

 terminal

```
$ python3 -m niko2 run variables.niko # (same file, continued)  
5
```

Niko says: Variable names can't have spaces — use one word like `high_score` or `playerName`. Pick names that explain themselves. Future-you will say thanks.

Try it yourself

Make a variable called `age` with your age in it. Then `say` it. Then add 1 to it and say it again — happy early birthday!

CHAPTER 4

Talking to the human: `ask`

Programs are more fun when they *listen*. `ask` shows a question on the screen, waits for the human to type an answer, and puts that answer in a variable:

```
ask_name.niko

ask "What is your name? " into name
say "Hello,", name
```

```
terminal

$ printf 'Sam\n' | python3 -m niko2 run ask_name.niko
What is your name? Hello, Sam
```

The little `into name` part says where the answer goes. Need a **number** instead of words? Use `ask number` — Niko will turn the typed answer into a real number so you can do math with it:

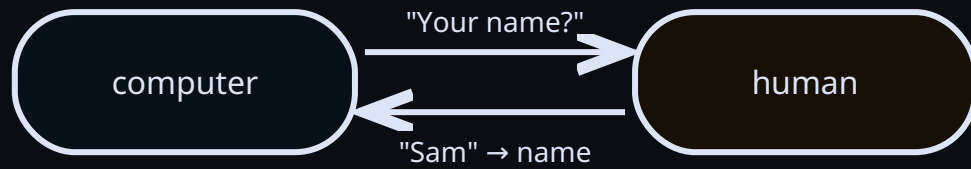
```
ask_age.niko

ask number "How old are you? " into age
set older to age + 5
say "In 5 years you will be", older
```

```
terminal

$ printf '30\n' | python3 -m niko2 run ask_age.niko
How old are you? In 5 years you will be 35
```

Niko says: If the human types something that isn't a number when you used `ask number`, Niko will ask again nicely. It won't crash — it just waits for a proper number.



`ask` is a conversation: a question goes out, an answer comes back.

CHAPTER 5

Making decisions: `if`

Sometimes the program should do different things in different situations. That's what `if` is for. Read it out loud — it sounds like English because it *is* almost English:

```
set score to 20
if score is 20:
    say "You got a perfect score!"
otherwise if score is 10:
    say "Halfway there!"
otherwise:
    say "Keep practicing!"
```

terminal

```
$ python3 -m niko2 run decisions.niko
You got a perfect score!
```



An `if` is a fork in the road. The program checks the question, then takes exactly one path.

For more than two choices, chain them with `otherwise if`. Niko checks from the top and runs the **first** one that's true:

decisions_chain.niko

```
set score to 10
if score is 20:
  say "You got a perfect score!"
otherwise if score is 10:
  say "Halfway there!"
otherwise:
  say "Keep practicing!"
```

terminal

```
$ python3 -m niko2 run decisions_chain.niko # score is 10 this time
Halfway there!
```

Niko says: Notice the words Niko uses for comparing: `is`, `is smaller than`, `is bigger than`. Say the line out loud — if it sounds right in English, it's probably right in Niko.

Try it yourself

Ask the human for their age with `ask number`. If they're younger than 13, say "Hi, fellow kid!". Otherwise if younger than 20, say "Hello, teenager!". Otherwise say "Greetings, grown-up!".

CHAPTER 6

Round and round: loops

Computers are wonderful at doing boring things many times without complaining. A **loop** tells Niko: "do this block again and again."

repeat — do it N times



repeat_loop.niko

```
repeat 3 times:  
  say "Hooray!"
```



terminal

```
$ python3 -m niko2 run repeat_loop.niko  
Hooray!  
Hooray!  
Hooray!
```

for each — visit everything in a list



for_each.niko

```
for each treat in ["ice cream", "pizza", "cake"]:  
  say "I love", treat
```

terminal

```
$ python3 -m niko2 run for_each.niko
I love ice cream
I love pizza
I love cake
```

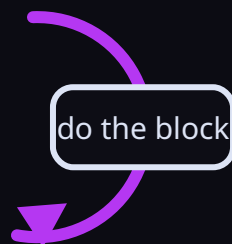
while — keep going until something changes

while_loop.niko

```
set n to 3
while n is bigger than 0:
  say n
  set n to n - 1
say "Blast off!"
```

terminal

```
$ python3 -m niko2 run while_loop.niko
3
2
1
Blast off!
```



← round again...
...until the test says sto

A loop is a circle: do the block, check the test, round again — until it's time to stop.

`stop` and `skip` — the escape hatches

`stop` jumps out of the loop completely. `skip` jumps to the next round, skipping the rest of this one:



stop_skip.niko

```
for each n in numbers 1 to 10:
  if n is 7:
    stop
  if n % 2 is 0:
    skip
  say n
```

 terminal

```
$ python3 -m niko2 run stop_skip.niko
1
3
5
```

Niko says: Careful with `while`: if the thing you're waiting for never happens, the loop spins *forever* and you'll have to stop the program yourself. Always make sure something inside the loop moves you closer to done!

Try it yourself

Use `repeat` to print the 5-times table (5, 10, 15...). Hint: keep a variable, add 5 each round. Then do it with `for each x in numbers 1 to 5` and multiply!

CHAPTER 7

Your own commands: functions

`say` and `ask` are commands that come with Niko. But you can **invent your own**! A **function** is a named bundle of instructions. You teach it once, then use it whenever you like:

functions.niko

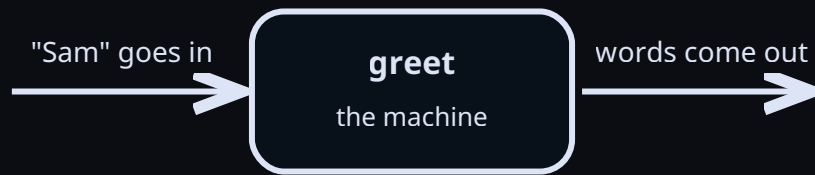
```
to greet with name: text:
  say "Hi,", name

to add with a: number, b: number -> number:
  give back a + b

greet("Sam")
greet("Alex")
say add(20, 22)
```

terminal

```
$ python3 -m niko2 run functions.niko
Hi, Sam
Hi, Alex
42
```



A function is a little machine: values go in, work happens, a result can come back out with `give back`.

The `with` part lists the **inputs** (also called *parameters*). `give back` sends a result back to whoever called the function. The `-> number` is a promise: "this function gives back a number."

Niko says: If you copy-paste the same code three times, that's a function begging to be born. Give it a name, teach it once, call it everywhere. Your programs get shorter *and* easier to understand.

Try it yourself

Write a function `to double with n: number -> number` that gives back twice the input. Then use it to double 21. (The answer to everything, doubled, is 84.)

CHAPTER 8

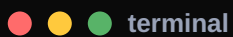
Shopping lists: lists

One variable holds one thing. A **list** holds a whole row of things, in order — like a shopping list:



lists.niko

```
set pets to ["dog", "cat", "hamster"]
say "I have", length(pets), "pets."
say "The second one is a", 2 of pets
```



```
$ python3 -m niko2 run lists.niko
I have 3 pets.
The second one is a cat
I love my dog
I love my cat
I love my hamster
```

Lists and loops are best friends. **for each** visits every item, one by one:



lists.niko

```
for each pet in pets:
  say "I love my", pet
```

terminal

```
$ python3 -m niko2 run lists.niko # (same file, continued)
I love my dog
I love my cat
I love my hamster
```

Niko says: Counting starts at **1** in Niko, not 0! The first item is **1 of my_list**, the second is **2 of my_list**. It matches how humans count — that's the point.

Try it yourself

Make a list of three animals. Use **for each** to say "I like ____" for each one. Then say how many animals there are with **length**.

CHAPTER 9

Little bundles: records

A list is great when the order matters. But what about a *person*? A person has a name *and* an age *and* a favourite colour — each piece has its own label. That's a **record**:



records.niko

```
set person to {"name": "Niko", "age": 2}  
say person["name"]  
say person.age  
say "Age:", person["age"]
```



terminal

```
$ python3 -m niko2 run records.niko  
Niko  
2  
Age: 2
```

person

name → "Ada"	pet → "cat"
age → 36	

A record keeps related facts together, each with its own name tag.

Try it yourself

Make a record for your favourite book: title, author, and pages. Say a sentence about it using all three fields.

CHAPTER 10

Playing with text

Niko comes with a **toy box** of ready-made text helpers. You borrow them with `import`, giving the box a short nickname:

text_tricks.niko

```
import "stdlib/text.niko" as text
say text.title("hello WORLD")
say text.capitalize("niko is fun")
say text.words("the quick brown fox")
```

terminal

```
$ python3 -m niko2 run text_tricks.niko
Hello World
Niko is fun
["the", "quick", "brown", "fox"]
```

The pattern is always the same: `nickname.helper("some text")`. Here are a few more favourites — guess what each does before you run it!

text_tricks2.niko

```
import "stdlib/text.niko" as text
say text.reverse_text("niko")
say text.pad_left("42", 5, "0")
say text.repeat("ha", 3)
```

terminal

```
$ python3 -m niko2 run text_tricks2.niko  
okin  
00042  
hahaha
```

Niko says: The full toy box is listed in [STDLIB.md](#) in the Niko folder. Whenever you think "someone must have already solved this" — look there first!

CHAPTER 11

Numbers and math

Niko does math the way you'd write it on paper:



math.niko

```
say 2 + 3 * 4
say 10 / 4
say sum([1, 2, 3, 4])
```



terminal

```
$ python3 -m niko2 run math.niko
14
2.5
10
6
10
```

And the toy box has math helpers too:



math.niko

```
import "stdlib/math.niko" as math
say math.gcd(12, 18)
say math.clamp(15, 0, 10)
```

terminal

```
$ python3 -m niko2 run math.niko # (same file, continued)
```

```
6
```

```
10
```

Try it yourself

You're throwing a party. There are 47 cupcakes and 8 friends (you included!). How many cupcakes does each person get, and how many are left over? (Hint: division gives one answer, but there's a sneakier question hiding here.)

CHAPTER 12

When things go wrong: errors

Here's a secret: **professional programmers see error messages all day long**. An error isn't failure — it's the computer telling you exactly what confused it. Let's make one on purpose:



oops.niko

```
say "Adding text and a number:"  
say "cookies: " + 3
```



terminal

```
$ python3 -m niko2 run oops.niko  
Adding text and a number:  
Niko error in oops.niko: Line 2: can only concatenate str (not "int")
```

Read it like a treasure map:

1. **What kind of problem?** The first line names it.
2. **Where?** The line number points at the exact spot.
3. **What did the computer expect?** The rest of the message is a hint.

Niko says: My debugging ritual: **read** the message, **find** the line, **fix one thing**, run again.
Never change five things at once — then you won't know which change helped!

Try it yourself

On purpose, write `say "hello` (forget the closing quote) and run it. Read the error.
Then fix it. Congratulations — you just debugged!

CHAPTER 13

Borrowing code: `use`

Big programs live in many files. `use` lets one file borrow the functions from another file — like borrowing a book from a shelf. First, the helper file with something useful in it:

```
helper.niko

to shout with word: text -> text:
  give back word + "!!!"
```

Now the main file borrows it:

```
main.niko

use "helper.niko"
say shout("hooray")
```

```
terminal

$ python3 -m niko2 run main.niko # inside the use_demo folder
hooray!!!
```

Niko says: `use` is for *your* files. `import` (which you met in Chapter 10) is for the built-in toy boxes like `stdlib/text.niko`. Same idea, different shelves!

CHAPTER 14

The toy box: the standard library tour

Niko ships with toy boxes — ready-made helpers written in Niko itself. You've already met the **text** box. Here's the whole shelf:



stdlib_tour.niko

```
import "stdlib/text.niko" as text
import "stdlib/math.niko" as math
import "stdlib/lists.niko" as lists
import "stdlib/records.niko" as records
import "stdlib/json.niko" as json
say text.reverse_text("niko")
say math.factorial(5)
say lists.take([1, 2, 3, 4], 2)
say records.record_keys({"a": 1, "b": 2})
say json.stringify({"pet": "cat"})
```



terminal

```
$ python3 -m niko2 run stdlib_tour.niko
okin
120
[1, 2]
["a", "b"]
{"pet": "cat"}
```

The deeper guide

<docs/niko-guide.html> — the full reference, exercises, and a 4-week plan

STDLIB.md

the complete list of every helper, in the Niko folder

Niko says: You don't need to memorize the toy box. Just remember it *exists*. When you need something, peek inside — that's what it's for.

Three machines, one language

Here's something special about Niko: the **same program** can run on **three different machines**:

The VM (`niko2 run`) — Niko's own little computer inside your computer. Fast to start, great for learning.

WebAssembly (`niko2 wasm --run`) — your program turned into code that runs in web browsers. That's what powers the playground!

Native (`niko2 native --run`) — your program turned into a real machine-code program, like the apps on your computer.

Watch — one file, three runs, same answer:



backends.niko

```
to cheer with times: number:
  for each n in numbers 1 to times:
    say "Hip hip!"

cheer(3)
say "Done!"
```

```
$ python3 -m niko2 run backends.niko
Hip hip!
Hip hip!
Hip hip!
Done!

$ python3 -m niko2 wasm backends.niko --run
  built backends.wasm (12921 bytes)
Hip hip!
Hip hip!
Hip hip!
Done!

$ python3 -m niko2 native backends.niko --run
  built backends (49816 bytes)
Hip hip!
Hip hip!
Hip hip!
Done!
```

Niko says: For this book, `niko2 run` is all you need. The other two are there when you're curious — or when you want your program to live on the web or run super fast.

CHAPTER 16

Mini projects: build three things

Time to put it all together! Three projects, easiest first. Each one uses things you've already learned — nothing new, just combined in fun ways.

Project 1 — The guessing game

The computer picks a secret number. You guess. It tells you "too big" or "too small" until you get it.

Uses: `ask number`, `while`, `if`.



game_guess.niko

```
set secret to 7
say "I am thinking of a number from 1 to 10."
ask number "Your guess: " into guess
while guess is not 7:
  if guess is smaller than 7:
    say "Too small!"
  otherwise:
    say "Too big!"
  ask number "Try again: " into guess
say "You got it!"
```



terminal

```
$ printf '3\n9\n7\n' | python3 -m niko2 run game_guess.niko
I am thinking of a number from 1 to 10.
Your guess: Too small!
Try again: Too big!
Try again: You got it!
```

Make it yours

Count the guesses and brag at the end ("You got it in 4 tries!"). Then let the *computer* guess *your* number — you say "bigger" or "smaller" and it narrows down. Trickier than it sounds!

Project 2 — The todo list

Keep a list of things to do, add new ones, and read the whole list back. Uses: **lists**, **for each**, functions.



todo.niko

```
set todos to ["Do my homework", "Tidy my room", "Feed the fish"]
say "My todo list:"
for each job in todos:
  say "-", job
set item 2 of todos to "Tidy my room (done!)"
say "Updated:"
say todos
```

 terminal

```
$ python3 -m niko2 run todo.niko
My todo list:
- Do my homework
- Tidy my room
- Feed the fish
Updated:
["Do my homework", "Tidy my room (done!)", "Feed the fish"]
```

Make it yours

Add a "done" feature: let the human cross an item off the list. (Hint: build a *new* list with only the items that aren't done yet.)

Project 3 — The quiz show

Two questions, a score, and a dramatic final reveal. Uses: `ask`, variables, `if`, functions.

 quiz.niko

```
set score to 0
ask "What is 2 + 2? " into answer1
if answer1 is "4":
    say "Correct!"
    set score to score + 1
otherwise:
    say "Oops, it is 4."
ask "What color is the sky? " into answer2
if answer2 is "blue":
    say "Correct!"
    set score to score + 1
otherwise:
    say "Oops, it is blue."
say "Your score:", score, "out of 2"
```

 terminal

```
$ printf '4\ngreen\n' | python3 -m niko2 run quiz.niko
What is 2 + 2? Correct!
What color is the sky? Oops, it is blue.
Your score: 1 out of 2
```

Make it yours

Add three more questions. Give a gold/silver/bronze medal based on the score with `otherwise if`. Add a victory dance (a `repeat` loop saying "" five times counts).

Niko says: You just built three real programs! Not "examples" — *programs*. The guessing game, the todo list, the quiz: these are the same ideas behind apps people use every day. You're thinking like a programmer now.

CHAPTER 17

Where to go next

You know the whole language now — really! Everything else is practice and new toy boxes. Here's your map:

The deeper guide

Full reference, 4-week plan, practice exercises, style guide

niko2 test

Write tests for your programs: `niko2 test` runs them and tells you what broke

niko2 get

The package manager — share your code and borrow other people's packages

niko2 fmt

The formatter — makes your code tidy automatically

All the examples, ready to run

Every program in this book lives in the `docs/beginners-examples/` folder. Take a look inside:

terminal

```
$ ls docs/beginners-examples
ask_age.niko decisions.niko for_each.niko hello.niko
ask_name.niko decisions_chain.niko functions.niko lists.niko
backends.niko game_guess.niko math.niko oops.niko
quiz.niko records.niko repeat_loop.niko stdlib_tour.niko
stop_skip.niko text_tricks.niko text_tricks2.niko todo.niko
use_demo variables.niko while_loop.niko
```

Each file is named after its chapter. Run any of them straight from the folder:

terminal

```
$ python3 -m niko2 run docs/beginners-examples/hello.niko  
Hello, world!
```

Ideas for your next programs

A dice roller — `ask number` how many dice, roll them with random numbers, add them up.

A story maker — ask for a name, a place, and a silly word, then tell a story with them.

A countdown timer — `while` loop with a wait, ending in "GO!"

Rock-paper-scissors — you versus the computer, best of five.

A grade calculator — ask for scores, say the average and a letter grade.

Niko says: The best way to learn programming is to build something *you* want to exist. Pick the idea above that made you smile and start. When you get stuck (you will — everyone does), re-read Chapter 12, fix one thing, and keep going. Happy coding!

CHAPTER 18

The niko2 toolbox

So far you've mostly used one command: `niko2 run`. But `niko2` is a whole toolbox. Here are the tools you'll reach for first — each one does one job, and does it well.

run — the starter

Runs your program. You know this one by heart now:

```
terminal
$ python3 -m niko2 run hello.niko
Hello, world!
```

check — the proofreader

`check` reads your program and points out mistakes *without* running it. Fast, safe, and great before you show your code to anyone:

```
terminal
$ python3 -m niko2 check docs/beginners-examples/hello.niko
✓ docs/beginners-examples/hello.niko: no type errors
```

fmt — the tidier

Messy spacing? Mixed-up quotes? `fmt` cleans your code automatically. Watch:

```
messy.niko (before)

say "Hello, world!"
```

```
terminal

$ python3 -m niko2 fmt messy.niko
formatted messy.niko
```

Now the file reads `say 'Hello, world!'` — single spaces, tidy quotes, same program.

Niko says: `niko2 fmt --check` only *reports* messy files without changing them — handy when you want to look before you tidy.

repl — the conversation

Type `niko2 repl` and Niko talks back, one line at a time. Perfect for trying an idea in five seconds:

```
terminal

$ python3 -m niko2 repl
Niko 2 REPL (VM). Type :help for help; :quit or Ctrl-D to leave.
niko> say "hi from repl"
hi from repl
```

test — the examiner

`niko2 test` runs the checks you've written for your program and tells you what passed and what broke. Professionals test everything — now you can too.

wasm and native — the translators

You met these two in Chapter 15: `niko2 wasm` turns your program into WebAssembly for browsers, and `niko2 native` builds a fast machine program through C.

get — the borrower

`niko2 get` fetches packages other people shared — extra toy boxes for your programs, installed with one command.

Niko says: The remaining tools — `build`, `disasm`, `init`, `info`, `deps`, `lock`, `publish`, `lsp`, `debug`, `migrate`, `fuzz` — are power tools. You'll meet them in the deeper guide when you're ready. Type `python3 -m niko2 --help` any time to see the full list.

The Niko Beginner's Guide — written with care for curious humans of all ages.
Every example in this book was run for real. If one misbehaves on your machine, that's a bug —
tell us!

Keep going: [the deeper guide](#) is waiting when you're ready.